

这个生产案例，好，很好，非常好。

why技术 4月21日

以下文章来源于码海，作者张国彬



码海

一起进阶，一起牛逼！



我们常说面试造火箭，很多人对此提出质疑，相信大家看了这篇文章会明白面试造火箭的道理。

这篇排查问题的技巧涉及到索引，GC，容器，网络抓包，全链路追踪等基本技能，没有这些造火箭的本事，排查这类问题往往会无从下手，本篇也能回答不少朋友的问题：

为什么学 Java 却要掌握网络，MySQL等其他知识体系，这会让你成为更出色的工程师哦。

一. 问题现象

商品团队反馈，会员部分 Dubbo 接口偶现超时异常，而且时间不规律，几乎每天都有，商品服务超时报错如下图：

```
com.alibaba.dubbo.remoting.transport.dispatcher.ChannelEventRunnable in run at line 57 (application)

Caused by TimeoutException
    Waiting server-side response timeout by scan timer, start time: 2019-09-12 17:22:45.683, end time: 2019-09-12 17:22:46.701, client elapsed: 0 ms, server elapsed: 1017 ms, timeout: 1000 ms, request: Request [id=174927, version=2.0.2, twoway=true, event=false, broken=false, data=RpcInvocation [methodName=getMemberDiscount, parameterTypes=[class java.lang.Long], arguments=[132487162], attachments=[path=com.beibeigroup.xretail.service.XretailMemberService, ticket=beibei-1f70dce4b12744acb0c218557223d112-7, in...

com.alibaba.dubbo.remoting.exchange.support.DefaultFuture in returnFromResponse at line 243 (application)
com.alibaba.dubbo.remoting.exchange.support.DefaultFuture in get at line 162 (application)
com.alibaba.dubbo.remoting.exchange.support.DefaultFuture in get at line 135 (application)
com.alibaba.dubbo.rpc.protocol.dubbo.DubboInvoker in doInvoke at line 95 (application)
com.alibaba.dubbo.rpc.protocol.AbstractInvoker in invoke at line 155 (application)
com.alibaba.dubbo.rpc.listener.ListenerInvokerWrapper in invoke at line 77 (application)
```

超时的接口平时耗时极短，平均耗时 4-5 毫秒。查看 Dubbo 的接口配置，商品调用会员接口超时时间是一秒，失败策略是 failFast，快速失败不会重试。

会员共部署了8台机器，都是 Java 应用，Java 版本使用的是 JDK 8，都跑在 Docker 容器中。

二. 问题分析

开始以为只是简单的接口超时，着手排查。首先查看接口逻辑，只有简单的数据库调用，封装参数返回。SQL 走了索引查询，理应返回很快才是。

于是搜索 Dubbo 的拦截器 ElapsedFilter 打印的耗时日志（ElapsedFilter 是 Dubbo SPI 机制的扩展点之一，超过 300 毫秒的接口耗时都会打印），这个接口的部分时间耗时确实很长。



再查询数据库的慢 SQL 平台，未发现慢 SQL。

1. 数据库超时？

怀疑是获取数据库连接耗时。代码中调用数据库查询可以分为两步，获取数据库连接和 SQL 查询，慢SQL 平台监测的 SQL 执行时间，既然慢 SQL 平台没有发现，那可能是建立连接耗时较长。

查看应用中使用了 Druid 连接池，数据库连接会在初始化使用时建立，而不是每次 SQL 查询再建立连接。Druid 配置的 initial-size（初始化链接数）是 5，max-active（最大连接池数量）是 50，但 min-idle（最小连接池数量）只有 1，猜测可能是连接长时间不被使用后都回收了，有新的请求进来后触发建立连接，耗时较长。

因此调整 min-idle 为 10，并加上了 Druid 连接池的使用情况日志，每隔 3 秒打印连接池的活跃数等信息，重新观察。

改动上线后，还是有超时发生，超时发生时查看连接池日志，连接数正常，连接池连接数也没什么波动。

2. STW？

因此重新回到报错日志观察，ElapsedFilter 打印的耗时日志已经消失。

由于在业务方法的入口和出口，以及数据库操作的前后打印了日志，发现整个业务操作耗时极短，都在几毫秒内完成。

但是调用端隔一段时间开始超时报错，且并不是调用一个接口超时，而是调用好几个接口都同时超时。

另外，调用端的几台机器都会同时报超时（可以在 elk 平台筛选 `hostname` 查看），而提供服务超时的机器是同一台。

这样问题就比较清晰了，**应该是某一时刻提供服务的某台机器出现比较全局性的问题导致几乎所有接口都超时**。

继续观察日志，抓了其中一个超时的请求从调用端到服务端的所有日志（理应有分布式 ID 可以追踪，`context id` 只能追踪单应用内的一个请求，跨应用就失效了，所以只能自己想办法，此处是根据调用IP+时间+接口+参数在 elk 定位）。

这次有了新的发现，服务端收到请求的时间比调用端发起调用的时间晚了一秒。

比较严谨的说法是调用端的超时日志中 Dubbo 有打印 `startTime`（记为T1）和 `endTime`，同时在服务端的接口方法入口加了日志，可以定位请求进来的时间（记为T2），比较这两个时间，发现 T2 比 T1 晚了一秒多，更长一点的有两到三秒。

内网调用网络时间几乎可以忽略不计，这个延迟时间极其不正常。很容易想到 Java 应用的 STW(Stop The World)，其中，特别是垃圾回收会导致短暂的应用停顿，无法响应请求。

排查垃圾回收问题第一件事自然是打开垃圾回收日志（GC log），GC log 打印了 GC 发生的时间，GC 的类型，以及 GC 耗费的时间等。增加 JVM 启动参数

```
-XX:+PrintGCDetails
-Xloggc:${APPLICATION_LOG_DIR}/gc.log
-XX:+PrintGCDateStamps
-XX:+PrintGCApplicationStoppedTime。
```

为了方便排查，同时打印了所有应用的停顿时间。（除了垃圾回收会导致应用停顿，还有很多操作也会导致停顿，比如取消偏向锁等操作）。由于应用是在 docker 环境，因此每次应用发布都会导致 GC 日志被清除，写了个上报程序，定时把 GC log 上报到 elk 平台，方便观察。

以下是接口超时时 GC 的情况：

```

2019-09-18T21:50:23.357+0800: 8291.179: Total time for which application threads were stopped: 0.0058146 seconds, Stopping threads took: 0.0015071 seconds
2019-09-18T21:50:26.429+0800: 8294.251: Total time for which application threads were stopped: 0.0181971 seconds, Stopping threads took: 0.0005323 seconds
2019-09-18T21:50:28.433+0800: 8296.255: Total time for which application threads were stopped: 0.0038281 seconds, Stopping threads took: 0.0004993 seconds
2019-09-18T21:50:32.893+0800: 8300.715: [GC (Allocation Failure) 2019-09-18T21:50:32.894+0800: 8300.715: [ParNew: 841434K->2961K(943744K), 1.9010708 secs] 1151594K->313165K(5138048K), 1.9024220 secs] [Times: user=6.25 sys=0.51, real=1.90 secs]
2019-09-18T21:50:34.796+0800: 8302.618: Total time for which application threads were stopped: 1.9082329 seconds, Stopping threads took: 0.0007156 seconds
2019-09-18T21:50:34.802+0800: 8302.624: Total time for which application threads were stopped: 0.0054011 seconds, Stopping threads took: 0.0005259 seconds
2019-09-18T21:50:34.806+0800: 8302.628: Total time for which application threads were stopped: 0.0035246 seconds, Stopping threads took: 0.0002232 seconds
2019-09-18T21:50:34.808+0800: 8302.630: Total time for which application threads were stopped: 0.0021907 seconds, Stopping threads took: 0.0001615 seconds
2019-09-18T21:50:34.810+0800: 8302.632: Total time for which application threads were stopped: 0.0019853 seconds, Stopping threads took: 0.0001600 seconds
2019-09-18T21:50:34.812+0800: 8302.634: Total time for which application threads were stopped: 0.0019172 seconds, Stopping threads took: 0.0001212 seconds
2019-09-18T21:50:34.817+0800: 8302.639: Total time for which application threads were stopped: 0.0043108 seconds, Stopping threads took: 0.0002131 seconds
2019-09-18T21:50:34.831+0800: 8302.652: Total time for which application threads were stopped: 0.0043300 seconds, Stopping threads took: 0.0006874 seconds

```

可以看到，有次 GC 耗费的时间接近两秒，应用的停顿时间也接近两秒，而且此次的垃圾回收是 ParNew 算法，也就是发生在新生代。所以基本可以确定，**是垃圾回收的停顿导致应用不可用，进而导致接口超时。**

2.1 安全点及 FinalReferecne

以下开始排查新生代垃圾回收耗时过长的问题。

首先，应用的 JVM 参数配置是

```

-Xmx5g
-Xms5g
-Xmn1g
-XX:MetaspaceSize=512m
-XX:MaxMetaspaceSize=512m
-Xss256k
-XX:SurvivorRatio=8
-XX:+UseParNewGC
-XX:+UseConcMarkSweepGC
-XX:CMSInitiatingOccupancyFraction=70

```

通过观察监控平台，发现堆的利用率一直很低，GC 日志甚至没出现过一次 CMS GC，由此可以排除是堆大小不够用的问题。

ParNew 回收算法较为简单，不像老年代使用的 CMS 那么复杂。

ParNew 采用标记-复制算法，把新生代分为 Eden 和 Survivor 区，Survivor 区分为 S0 和 S1，新的对象先被分配到 Eden 和 S0 区，都满了无法分配的时候把存活的对象复制到 S1 区，清除剩下的对象，腾出空间。

比较耗时的操作有三个地方，**找到并标记存活对象，回收对象，复制存活对象**（此处涉及到比较多垃圾回收的知识，详细部分就不展开说了）。

找到并标记存活对象前 JVM 需要暂停所有线程，这一步并不是一蹴而就的，需要等所有的线程都跑到安全点。

部分线程可能由于执行较长的循环等操作无法马上响应安全点请求，JVM 有个参数可以打印所有安全点操作的耗时，加上参数

```
-XX:+PrintSafepointStatistics
-XX:PrintSafepointStatisticsCount=1
-XX:+UnlockDiagnosticVMOptions
-XX:-DisplayVMOutput
-XX:+LogVMOutput
-XX:LogFile=/opt/apps/logs/safepoint.log。
```

```
vmop [threads: total initially_running wait_to_block] [time: spin block sync cleanup vmop] page_trap_count
8280.528: RevokeBias [ 1872 0 0 ] [ 0 0 0 2 0 ] 0
vmop [threads: total initially_running wait_to_block] [time: spin block sync cleanup vmop] page_trap_count
8291.173: RevokeBias [ 1877 0 0 ] [ 0 0 1 3 0 ] 0
vmop [threads: total initially_running wait_to_block] [time: spin block sync cleanup vmop] page_trap_count
8294.232: BulkRevokeBias [ 1878 0 0 ] [ 0 0 0 2 14 ] 0
vmop [threads: total initially_running wait_to_block] [time: spin block sync cleanup vmop] page_trap_count
8296.251: no vm operation [ 1865 0 0 ] [ 0 0 0 2 0 ] 0
vmop [threads: total initially_running wait_to_block] [time: spin block sync cleanup vmop] page_trap_count
8300.710: GenCollectForAllocation [ 1867 0 0 ] [ 0 0 0 4 1902 ] 0
vmop [threads: total initially_running wait_to_block] [time: spin block sync cleanup vmop] page_trap_count
8302.619: RevokeBias [ 1867 0 8 ] [ 0 0 0 4 0 ] 0
vmop [threads: total initially_running wait_to_block] [time: spin block sync cleanup vmop] page_trap_count
8302.624: RevokeBias [ 1866 0 0 ] [ 0 0 0 2 0 ] 0
vmop [threads: total initially_running wait_to_block] [time: spin block sync cleanup vmop] page_trap_count
8302.628: RevokeBias [ 1866 0 1 ] [ 0 0 0 1 0 ] 0
```

安全点日志中 spin+block 时间是线程跑到安全点并响应的的时间，从日志可以看出，此处耗时极短，大部分时间都消耗在 vmop 操作，也就是到达安全点之后的操作时间，排除线程等待进入安全点耗时过长的可能。

继续分析回收对象的耗时，根据引用强弱程度不同，Java 的对象类型可分为各类 Reference。

其中FinalReference 较为特殊，对象有个 finalize 方法，可在对象被回收之前调用，给对象一个垂死挣扎的机会。有些框架会在对象 finalize 方法中做一些资源回收，关闭连接的操作，导致垃圾回收耗时增加。

因此通过增加JVM参数 -XX:+PrintReferenceGC，打印各类 Reference 回收的时间。

```
2019-09-19T06:12:24.419+0800: 28877.836: Total time for which application threads were stopped: 0.0194426 seconds, Stopping threads took: 0.0006113 seconds
2019-09-19T06:12:25.805+0800: 28879.222: Total time for which application threads were stopped: 0.0113319 seconds, Stopping threads took: 0.0005967 seconds
2019-09-19T06:13:46.738+0800: 28900.146: Total time for which application threads were stopped: 0.0040840 seconds, Stopping threads took: 0.0005429 seconds
2019-09-19T06:14:22.500+0800: 28934.387: GC (Allocation Failure) 2019-09-19T06:14:22.500+0800: 28934.389: [PermGen:2019-09-19T06:14:22.500+0800: 28935.916: [SoftReference, 0 refs, 0.0000000 secs]2019-09-19T06:14:22.500+0800: 28935.916: [WeakReference, 145 refs, 0.0000439 s
4232019-09-19T06:14:22.500+0800: 28935.926: [FinalReference, 42 refs, 0.0000000 secs]2019-09-19T06:14:22.500+0800: 28935.926: [PhantomReference, 0 refs, 1 refs, 0.0000000 secs]2019-09-19T06:14:22.500+0800: 28935.926: [JNI Weak Reference, 0.0000000 secs]: 8411558-46794C9
437440), 1.6862816 secs] 1149387X-31871K(5138848K), 1.6100910 secs] [Times: user=5.88 sys=0.64, real=1.61 secs]
2019-09-19T06:14:22.502+0800: 28935.928: Total time for which application threads were stopped: 1.6178280 seconds, Stopping threads took: 0.0016488 seconds
2019-09-19T06:14:22.511+0800: 28935.928: Total time for which application threads were stopped: 0.0023957 seconds, Stopping threads took: 0.0004895 seconds
2019-09-19T06:14:22.514+0800: 28935.931: Total time for which application threads were stopped: 0.0023321 seconds, Stopping threads took: 0.0003813 seconds
2019-09-19T06:14:22.517+0800: 28935.933: Total time for which application threads were stopped: 0.0022120 seconds, Stopping threads took: 0.0002424 seconds
2019-09-19T06:14:22.519+0800: 28935.935: Total time for which application threads were stopped: 0.0020048 seconds, Stopping threads took: 0.0001736 seconds
```

可以看到，FinalReference 的回收时间也极短。

最后看复制存活对象的耗时，复制的时间主要由存活下来的对象大小决定，从 GC log 可以看到，每次新生代回收基本可以回收百分之九十以上的对象，存活对象极少。因此基本可以排除这种可能。

问题的排查陷入僵局，ParNew 算法较为简单，因此 JVM 并没有更多的日志记录，所以排查更多是通过经验。

2.2 垃圾回收线程

不得已，开始求助网友，上 stackoverflow 发帖（链接见文末），有大神从 GC log 发现，有两次 GC 回收的对象大小几乎一样，但是耗时却相差十倍，因此建议确认下系统的 CPU 情况，是否是虚拟环境，可能存在激烈的 CPU 资源竞争。

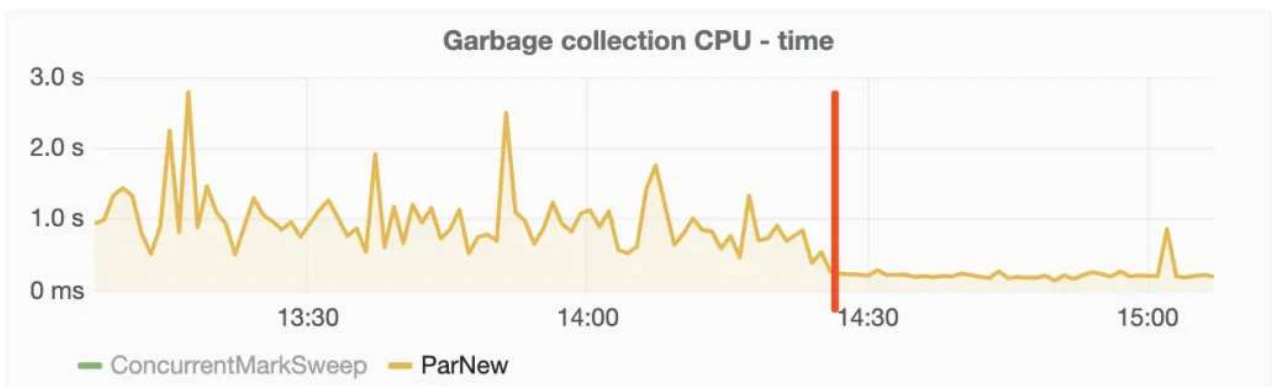
其实之前已经怀疑是 CPU 资源问题，通过监控平台发现，垃圾回收时 CPU 波动并不大。

但运维发现我们应用的垃圾回收线程数有些问题（GC 线程数可以通过 jstack 命令打印线程堆栈查看），JVM 的 GC 线程数量是根据 CPU 的核数确定的，如果是八个核心以下，GC 线程数等于 CPU 核心数，我们应用跑在 docker 容器，分配的核心是六核，但是新生代 GC 线程数却达到了 53 个，这明显不正常。

最后发现，这个问题是 JVM 在 docker 环境中，获取到的 CPU 信息是宿主机的（容器中 /proc 目录并未做隔离，各容器共享，CPU 信息保存在该目录下），并不是指定的六核心，宿主机是八十核心，因此创建的垃圾回收线程数远大于 docker 环境的 CPU 资源，导致每次新生代回收线程间竞争激烈，耗时增加。

通过指定 JVM 垃圾回收线程数为 CPU 核心数，限制新生代垃圾回收线程，增加 JVM 参数

```
-XX:ParallelGCThreads=6  
-XX:ConcGCThreads=6
```



效果立竿见影！

新生代垃圾回收时间基本降到 50 毫秒以下，成功解决垃圾回收耗时较长问题。

三. 问题重现

本以为超时问题应该彻底解决了，但还是收到了接口超时的报警。现象完全一样，同一时间多个接口同时超时。查看 GC 日志，发现已经没有超过一秒的停顿时间，甚至上百毫秒的都没有。

1. Arthas 和 Dubbo

回到代码，重新分析。

开始对整个 Dubbo 接口接收请求过程进行拆解，打算借助阿里巴巴开源的 Arthas 对请求进行 trace，打印全链路各个步骤的时间。

服务端接收 dubbo 请求，从网络层开始再到应用层。具体是从 netty 的 worker 线程接收到请求，再投递到 dubbo 的业务线程池（应用使用的是 ALL 类型线程池），由于涉及到多个线程，只能分两段进行 trace，netty 接收请求的一段，dubbo 线程处理的一段。

Arthas 的 trace 功能可以在后台运行，同时只打印耗时超过某个阈值的链路。（trace 采用的是 instrument+ASM，会导致短暂的应用暂停，暂停时间取决于被植入的类的数量，需注意）

由于没有打印参数，无法准确定位超时的请求，而且 trace 只能看到调用第一层的耗时时间，结果都是业务处理时间过长，最后放弃了该 trace 方法。

受 GC 线程数思路的启发，由于应用运行基本不涉及刷盘的操作，都是 CPU 运算+内存读取，耗时仍旧应该是线程竞争或者锁引起的。

重新回到 jstack 的堆栈进行排查，发现总线程数仍有 900+，处于一个较高的水平，其中 forkjoin 线程和 netty 的 worker 的线程数仍较多。

于是重新搜索代码中线程数是通过 CPU 核心数设置（`Runtime.getRuntime().availableProcessors()`）的地方，发现还是有不少框架使用这个参数，这部分无法通过设置 JVM 参数解决。

因此和容器组商量，是否能够给容器内应用传递正确的 CPU 核心数。

容器组确认可以通过升级 JDK 版本和开启 CPU SET 技术限制容器使用的 CPU 数，从 Java 8u131 和 Java 9 开始，JVM 可以理解和利用 cpuset 来确定可用处理器的大小([Java 和 cpuset 见文末参考链接](#))。

现在使用的也是JDK 8，小版本升级风险较小，因此测试没问题后，推动应用内8台容器升级到了 Java 8u221，并且开启了 cpu set，指定容器可使用的 CPU 数。

重新修改上线后，还是有超时情况发生。

2. 网络抓包

观察具体的超时机器和时间，发现并不是多台机器超时，而是一段期间有某台机器比较密集的超时，而且频率比之前密集了许多。

以前的超时在各机器之间较为随机，而且频率低很多。

比较奇怪的是，应用 经过发布之后，原来超时的机器不超时了，而是其他的机器开始超时，看起来像是接力赛。（docker 应用的宿主机随着每次发布都可能变化）

面对如此奇怪的现象，只能怀疑是宿主机或者是网络问题了，因此开始网络抓包。

由于超时是发生在一台机器上，而且频率较为密集，因此直接进入该机器，使用 tcpdump 命令抓取host 是调用方进来的包并保存抓包记录，通过 wireshark 分析（wireshark 是抓包分析神器），抓到了一些异常包。

4454	251.637427	10.54.103.93	10.54.201.140	TCP	1453	3257 → 20880 [PSH, ACK] Seq=1573886 Ack=324260 Win=3254 Len=1399
4455	251.637462	10.54.201.140	10.54.103.93	TCP	54	20880 → 3257 [ACK] Seq=324260 Ack=1575285 Win=460 Len=0
4456	251.638859	10.54.201.140	10.54.103.93	TCP	254	20880 → 3257 [PSH, ACK] Seq=324260 Ack=1575285 Win=460 Len=200
4457	251.857657	10.54.201.140	10.54.103.93	TCP	254	[TCP Retransmission] 20880 → 3257 [PSH, ACK] Seq=324260 Ack=1575285 Win=460 Len=
4458	252.289661	10.54.201.140	10.54.103.93	TCP	254	[TCP Retransmission] 20880 → 3257 [PSH, ACK] Seq=324260 Ack=1575285 Win=460 Len=
4459	252.331852	10.54.103.93	10.54.201.140	TCP	413	3257 → 20880 [PSH, ACK] Seq=1575285 Ack=324460 Win=3254 Len=359
4460	252.331867	10.54.103.93	10.54.201.140	TCP	415	3257 → 20880 [PSH, ACK] Seq=1575644 Ack=324460 Win=3254 Len=361
4461	252.331882	10.54.103.93	10.54.201.140	TCP	60	[TCP Dup ACK 4459#1] 3257 → 20880 [ACK] Seq=1576005 Ack=324460 Win=3254 Len=0
4462	252.331886	10.54.103.93	10.54.201.140	TCP	413	[TCP Out-of-Order] 3257 → 20880 [PSH, ACK] Seq=1575285 Ack=324460 Win=3254 Len=3
4463	252.331895	10.54.201.140	10.54.103.93	TCP	54	20880 → 3257 [ACK] Seq=324460 Ack=1576005 Win=459 Len=0
4464	252.332081	10.54.103.93	10.54.201.140	TCP	413	[TCP Out-of-Order] 3257 → 20880 [PSH, ACK] Seq=1575285 Ack=324460 Win=3254 Len=3
4465	252.332090	10.54.201.140	10.54.103.93	TCP	54	[TCP Window Update] 20880 → 3257 [ACK] Seq=324460 Ack=1576005 Win=460 Len=0
4466	252.332093	10.54.103.93	10.54.201.140	TCP	60	[TCP Dup ACK 4459#2] 3257 → 20880 [ACK] Seq=1576005 Ack=324460 Win=3254 Len=0
4467	252.332278	10.54.103.93	10.54.201.140	TCP	413	3257 → 20880 [PSH, ACK] Seq=1576005 Ack=324460 Win=3254 Len=359
4468	252.332290	10.54.201.140	10.54.103.93	TCP	54	20880 → 3257 [ACK] Seq=324460 Ack=1576364 Win=460 Len=0
4469	252.333025	10.54.201.140	10.54.103.93	TCP	313	20880 → 3257 [PSH, ACK] Seq=324460 Ack=1576364 Win=460 Len=250

注意抓包中出现的「TCP Out_of_Order」,「TCP Dup ACK」,「TCP Retransmission」，三者的释义如下:

TCP Out_of_Order：一般来说是网络拥塞，导致顺序包抵达时间不同，延时太长，或者包丢失，需要重新组合数据单元，因为他们可能是由不同的路径到达你的电脑上面。

TCP dup ack XXX#X：重复应答，#前的表示报文到哪个序号丢失，#后面的是表示第几次丢失

TCP Retransmission：超时引发的数据重传

终于，通过在 elk 中超时日志打印的 contextid，在 wireshark 过滤定位超时的 TCP 包，发现超时的时候有丢包和重传。

抓了多个超时的日志都有丢包发生，确认是网络问题引起。

问题至此基本告一段落，接下来容器组继续排查网络偶发丢包问题。

3. 巨人的肩膀

- stackoverflow 提问帖子 : <http://aakn.cn/YbFsl>
- jstack 可视化分析工具 : <https://fastthread.io/>
- 2018年的 Docker 和 JVM : <https://www.jdon.com/51214>

why哥说

前面就是公众号[码海]的号主坤哥给大家分享的一个线上问题排查案例。建议大家关注一波坤哥，他某电商独角兽大厂技术专家，发布的文章都很硬核。文首可以直达该公众号。

不知道大家读这个案例的时候是什么感觉，反正我整体读下来之后就一个字：通透！

从最开始 Dubbo 调用超时的这个表象，分别从数据库、GC、网络、链路追踪等各个角度去分析了问题，且是一个循序渐进的过程。

就是说大家的排查套路都无外乎这样，层层递减的排查，抽丝剥茧的寻找问题。

而且这里面还涉及到了很多其他的知识点，由于不是本文的重点，所以作者没有详细的写出来。

比如举几个例子：

1. 为什么要说“失败策略是 failFast，快速失败不会重试”？因为如果是 failover，会默认重试，且超时时间是重试时间之和。所以，他告诉我们，这里没有重试，超时不是因为请求重试带来的时间叠加导致的。
2. 文章提到的 ElapsedFilter 过滤器，“超过 300 毫秒的接口耗时都会打印”，是作者公司自己扩展的 Filter，基于 Dubbo 的 SPI 实现的，并不是 Dubbo 官方的自带功能。所以，他才额外提了一句“ElapsedFilter 是 Dubbo SPI 机制的（自定义）扩展点之一”。

3. 作者用的 **Druid** 连接池，猜测连接长时间不被使用后都回收了，那么关于 **Druid** 的配置文件中的有关时间的配置，你是否知道且清楚其作用？
4. 如果要观察 **GC** 日志，你是否大概知道应该配置什么参数，是否知道应该关注的信息是什么？为什么他这里要提到安全点？安全点和 **STW** 的时间之间的关系又是什么？
5. 等等后面的一些关于容器的、**Arthas**工具使用的、网络抓包工具使用的相关技能和知识储备。

正如文章开头说的，当把这些知识单独拎出来形成面试题的时候，也许你会觉得，为什么你老是问我 **MySQL** 的知识、问我网络相关的知识、问我一些用不上的垃圾回收的知识？

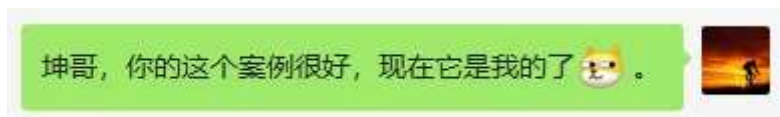
问你，把你问的哑口无言不是目的。考察你知识的广度，让你学以致用才是目的。重要的是把你学的一个个孤立的点，通过某种方式，串联起来。

而这个案例分享出来，你发现了没有，完全没有背景相关的铺垫，你看起来却毫无障碍。这就是前面说的“排查套路”，所以，套路在这里，背景你可以自己去填充。

填充，懂吧？就是适当的加工。

面试的时候万一能用上呢？把“拿来主义”贯彻到底。

就像我：你这个案例很好，现在它是我的了。



最后，不得不补充一句，有的同学的解决方案，可能就是把超时时间从 1 秒改到了 2 秒，甚至更长，然后问题就解决了.....

也不能说不行，只是可惜错过了一次排查的机会。

推荐 ：[哎，这要人老命的缓存一致问题啊!!!](#)

推荐 ：[当我看技术文章的时候，我在想什么？](#)

推荐 ：[真是绝了！这段被JVM动了手脚的代码！](#)

推荐 ：[面试官一个线程池问题把我问懵逼了。](#)

推荐 ：[这个Bug的排查之路，真的太有趣了。](#)

我是 **why**，一个主要写代码，经常写文章，偶尔拍视频的程序猿。

欢迎关注我呀。



why技术

一个主要写代码，经常写文章，偶尔拍视频的风骚程序猿。
156篇原创内容

公众号

喜欢此内容的人还喜欢

当面试官问你这个问题的时候，想听到什么？

why技术

复旦校园江湾一夜，青年志愿者很靠谱

中国青年志愿者

前戏怎么玩，女友更满意？

草莓爱问